

Agent Coding Introduction

Set up the environment, stand up an agent, take your first scrape

Shihao Yang • Candice Djorno

July 20, 2026

SISMID 2026 • Day 1, 9:45

What this first hour is for

Mauricio just gave you the big picture. Now we get your hands on a keyboard.

By the end of this session everyone in the room will have:

1. an **identical coding environment**, with nothing installed locally,
2. a working **AI coding agent** (Codex, Claude Code, or Antigravity CLI) you set up yourself,
3. and your **first scraped data stream**: real numbers pulled from the open web, plotted and sanity-checked.

The promise

You will not write a data-collection platform. You will describe what you want, the agent (or a pre-filled notebook) fetches it, and you keep the judgment.

One environment for everyone

The room we are teaching

Keep it streamlined

- one bad install can cost an hour
- wants a result to keep

Lightweight and ready for non-coder

- done in five minutes
- wants the real data plumbing

Two moves serve both

An **identical environment** removes the setup lottery. An **agent** lifts the novice to a working result and frees the expert from boilerplate. The agent is the equalizer we lean on all week.

Pick a lane to get the environment

Same repository, three ways in. Codespaces is the supported path.

- **GitHub Codespace (recommended).** A preconfigured container in your browser. Nothing to install. Free on your own account.
- **Clone and run locally.** If you prefer your own machine: `pip install -r requirements.txt` on Python 3.12.
- **Google Colab.** Open the notebooks with just a Google account.

Launch a Codespace (do this now)

github.com/Shihao-Yang/sismid2026 → green Code button → Codespaces → Create codespace on main. Builds in about a minute.

Confirm it works: the smoke test

Before anything else, prove the environment is healthy.

- Open `notebooks/00_smoke_test.ipynb` and run all cells, or in a terminal run `python scripts/smoke_test.py`.
- You should see library versions and **“Environment OK”**.

*If the smoke test passes, the rest of the week has no “it works on my laptop” problem.
That is the whole point of a shared container.*

Standing up a coding agent

What is a coding agent?

A coding agent is an LLM that can **read your files, run code, see the error, and try again**, inside your project, from a plain-English request.

OpenAI Codex

- CLI, runs in the Codespace terminal

Claude Code

- CLI, runs in the Codespace terminal

Antigravity CLI

- CLI (agy), free Google login

All three are command-line agents: one script installs them and they run right in the browser Codespace. We set up all three so you can compare them on the same task.

Not autocomplete. A junior collaborator that does the typing and the debugging while you keep the judgment.

Install and authenticate (together, in class)

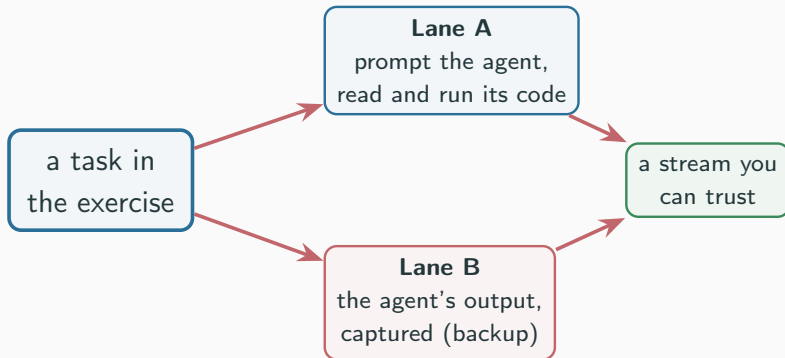
The container ships *no* agent on purpose, so you learn to do this yourself.

```
# 1. install all three CLIs
bash scripts/setup-agents.sh

# 2. log in (class passcode announced in class)
bash scripts/codex-login.sh # Codex
source scripts/claude-login.sh # Claude Code
agy # Antigravity: your own Google login
```

- source for Claude (it exports a token into *this* shell); bash for Codex (it just writes `~/.codex/auth.json`).
- You are identified by your GitHub username, so a Codespace restart hands you back the *same* credential.
- **Golden rule:** keys never get committed and never get pasted into a notebook cell.
- Sanity check: “list the CSV files in `data/` and print the first rows of each.”

Two lanes, one destination



- Agent not authenticated yet? Use Lane B and come back. Nobody is blocked.
- New to this? Pair with a neighbor: an expert partner plus the agent is the fastest way to learn what good output looks like.
- TA and instructors circulate. Raise a hand early, not after an hour.

Your first scrape

“Scraping” just means fetching web data into a table you can model. We are going to do it **right now**, together, and you follow along.

The goal on screen in the next five minutes

Dengue search interest for **Mexico**, the **past 5 years**, as a tidy, reproducible CSV that ends on *this week*. Plus the historical 2004-2011 window that feeds the 11:00 model.

We do it two ways: first by hand on the website, then by handing one prompt to an agent. Watch the difference.

The old way: open the website

Open

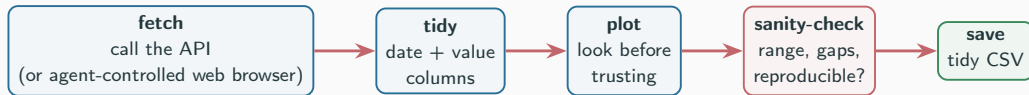
trends.google.com → search **dengue** → region **Mexico** → past **5 years**.

You get a clean 0-100 curve you can eyeball. But for real work it fights you:

- values **rescale** the moment you add or drop a term,
- you can compare at most **5 terms** at once,
- getting it into code, **reproducibly**, means clicking export every time.

Fine for a look, hopeless as a pipeline. So let's have an agent script exactly this, dengue being the backbone signal we model at 11:00.

The loop that every stream will use



Learn it once here. Google Trends, wastewater, mobility, and news all reuse the exact same five steps.

Now let an agent do it

Same goal, handed to an agent instead of your mouse. Pick one and open it:

Open (pick one)

- **Codex:** type `codex` in the Codespace terminal
- **Claude Code:** type `claude` in the Codespace terminal
- **Antigravity CLI:** type `agy` in the Codespace terminal

All three take the same plain-English goal and run in the same terminal. Pick whichever you like.

Give it one prompt

Prompt (paste this)

```
Pull Google Trends for a handful of dengue
terms in Mexico: "dengue", "síntomas de dengue",
"mosquito", over the past 5 years. Return a
tidy date + one-column-per-term DataFrame. Plot the three series
and print the date of the last point.
```

One paragraph. No API keys, no boilerplate. This is the whole live flow.

What comes back

The agent writes `gt_fetch(...)`, runs it, and hands you:

Result (a real run, July 2026)

262 weekly points $\times$ 3 terms, Jul 2021 to **the week of Jul 12–18, 2026**. The last point is *this week*: the pull is genuinely live.

- A plot of the three dengue terms appears, current through today.
- The exact code it wrote is captured in the **Lane B** notebook, so a rate-limited neighbor sees the same thing.

From a sentence to a live, reproducible signal. That is the point.

Verify what came back (your job, not the agent's)

Every scraped stream earns a few cheap checks before it enters a model:

- **Range and gaps:** does the date span match what you asked? Missing months quietly become wrong models.
- **Scale sanity:** plausible values, not all zeros or a flat line from a failed pull.
- **Reproducibility:** pull twice. Because Google samples searches, repeat pulls can *differ* (a little on the public 0-100 endpoint, a lot on the raw Health Trends API). That is what this afternoon's preprocessing fixes.
- **Face validity:** do the peaks land in a plausible Mexican dengue season?

The habit to build

One plot and one sanity question per stream. This is the skill, more than the fetch.

When Google Trends fights back (it will)

A live pull often hits a **429 (too many requests)**, then succeeds on the next try. That is normal. Two things keep you moving:

- **Retry.** The notebook waits and retries automatically. We tested this from a Codespace on an **Azure** datacenter IP: 429, wait, then success. Usually enough.
- **The cache (Lane B).** If it still will not budge, the notebook falls back to the cached CSV, so you are never stuck.

*If a block is ever really stubborn, there is a proper fix: routing through a proxy or VPN.
We cover that this afternoon, in the data-scraping session.*

Now you: Exercise 0

Open `exercise/exercise0_data_scraping` and work either lane.

- Set up the environment and pass the smoke test.
- Stand up an agent (or use Lane B).
- Scrape dengue search interest for Mexico, add related terms, sanity-check, save a tidy CSV.
- Discuss: why might search interest rise without any real change in disease activity, and what do you do about two pulls disagreeing?

What to walk away with

The dengue search CSV you scraped yourself (ready for 11:00), and the fetch-tidy-plot-check-save reflex. Swap in your own country and disease if you finish early.

- **Next (11:00):** the Dengue exercise. Intro to Google Dengue Trends, then map the search series you just scraped onto dengue cases with least squares.
- **This afternoon:** Google Trends in depth, where the same scrape returns different numbers each time and needs statistical preprocessing.
- **All week:** the agent stays your coding tool, and Lane B is always there.

You now have an environment, an agent, and a scraped dengue signal. That is the entire foundation the rest of the course builds on.

Questions?

Repo: `github.com/Shihao-Yang/sismid2026`